

TASK 03 — Transactions & Distributed Transaction Fundamentals

Step 1 — Study ACID

ACID describes the four important properties that make database transactions reliable.

1. A → Atomicity

Atomicity means: “All or Nothing.”

A transaction contains multiple database operations. Either **all operations succeed**, or **all are rolled back**.

Example:

Create Order



Create Order Items



Update Inventory

If inventory update fails, the order and order items should also be cancelled.

Create Order 

Create Items 

Update Inventory 



ROLLBACK



Nothing is saved

2. C → Consistency

Consistency means the database must remain valid before and after a transaction.

For example, suppose a product has:

Stock = 10

Customer orders 3 items:

Stock = 10 → 7

The database should never end up with invalid data such as:

Stock = -50

Database constraints and business rules help maintain consistency.

3. I → Isolation

Isolation means one transaction should not incorrectly interfere with another transaction.

Suppose two customers order the same product at the same time.

Customer A → Order 3

Customer B → Order 4

If only 5 items are available, the database needs to properly coordinate these transactions so that incorrect inventory doesn't occur.

4. D → Durability

Durability means once a transaction is successfully committed, the data is permanently saved.

Example:

Order Created



Transaction COMMIT



Database stores order

Even if the application/server crashes immediately afterward, the committed order should remain in the database.

Step 2 — Understand Local Transactions

These operations are related to one business operation: **placing an order**.

Example:

Initial inventory

Product: Laptop

Stock: 10

Customer orders 2 laptops.

The application performs:

1. Create Order
2. Create Order Items
3. Reduce inventory from 10 → 8

All three operations should normally belong to **one database transaction**.

Imagine this happens:

Create Order 

Create Order Items 

Update Inventory 

Without a transaction, the database could contain an order for a product whose inventory was never reduced.

That's an inconsistent state.

With a transaction:

BEGIN TRANSACTION

Create Order

Create Order Items

Update Inventory

COMMIT

If something fails:

BEGIN TRANSACTION

Create Order 

Create Order Items 

Update Inventory 

ROLLBACK

The previous changes are undone.

Step 3 — Implement Spring Transaction Management

Usually, we place it on the **service-layer method** that represents the complete business operation.

Example

```
@Service

public class OrderService {

    @Transactional

    public void createOrder(Order order) {

        // 1. Create order

        orderRepository.save(order);

        // 2. Create order items

        orderItemRepository.saveAll(order.getItems());

        // 3. Update inventory

        inventoryService.updateInventory(order);

    }

}
```

The important point is that all these operations are treated as **one transaction**.

What happens internally

When this method starts:

```
@Transactional

public void createOrder(Order order) {

    Spring starts a database transaction.

    createOrder()
```



BEGIN TRANSACTION

|— Save Order

|— Save Order Items

└— Update Inventory



SUCCESS?

/ \

YES NO

| |

COMMIT ROLLBACK

If everything succeeds

COMMIT

All changes are permanently stored.

If an appropriate exception causes rollback

ROLLBACK

The transaction's changes are undone.

Step 4 — Test Successful Transaction

Example, after a successful order:

Orderbtable

id	customer
----	----------

101	Rahul
-----	-------

Order Items

order_id	product	quantity
----------	---------	----------

101	Laptop	2
-----	--------	---

Inventory

product	stock
---------	-------

Laptop	8
--------	---

All three changes should exist.

Therefore:

Order created

Order items created

Inventory updated

Transaction committed

Simple Real-World Example

Imagine you are buying a phone.

Initially:

Phone Stock = 5

You purchase 1 phone.

The application performs:

Step 1 → Create Order

Step 2 → Create Order Item

Step 3 → Reduce Stock

After success:

Phone Stock = 4

If Step 3 fails:

Create Order 

Create Order Item 

Reduce Stock 

Spring can roll back the transaction:

ROLLBACK

So the database returns to its previous state:

Order → Not created

Order Item → Not created

Stock → 5

Step 5 — Test Transaction Failure

This step checks whether **rollback works correctly** when something goes wrong during a database transaction.

What does it mean

We intentionally make one database operation fail **after an earlier operation has already succeeded**.

For example:

Start Transaction



Create Order 



Create Order Item  Failure



ROLLBACK

Because of **Atomicity**, the successful operation should also be undone.

Example

Suppose we have:

Order ID = 101

The application first saves the order:

Create Order → SUCCESS

Then we intentionally generate an error:

```
orderRepository.save(order);
```

```
// Intentionally create an error
```

```
throw new RuntimeException("Testing transaction failure");
```

If the method is:

@Transactional

```
public void createOrder(Order order) {  
    orderRepository.save(order);  
    throw new RuntimeException("Testing transaction failure");  
}
```

Spring detects the runtime exception and normally performs:

ROLLBACK

So after the method finishes:

Order saved? → ❌ No

The database returns to its previous state.

Before transaction

Orders table

ID	Customer
----	----------

100	Ravi
-----	------

During transaction

101	Rahul	← temporarily created
-----	-------	-----------------------

Failure occurs

RuntimeException

↓

ROLLBACK

After rollback

Orders table

ID	Customer
----	----------

100	Ravi
-----	------

Order **101** is removed.

Step 6 — Understand Transaction Boundaries

A **transaction boundary** defines which database operations belong to the same transaction.

Case 1 — Order Service + Order DB

Order Service



Order DB

Here, the Order Service communicates with its own database.

For example:

Order Service



@Transactional



Order DB

This can normally be handled as **one local database transaction**.

Example

@Transactional

```
public void createOrder(Order order) {  
    orderRepository.save(order);  
    orderItemRepository.saveAll(order.getItems());  
}
```

Both operations use the **Order DB**.

If the second operation fails:

Save Order 

Save Order Item 



ROLLBACK

The first operation can also be rolled back.

Case 2 — Order Service → Payment Service → Payment DB

Order Service



Payment Service



Payment DB

This is different.

There are now **two separate services** and potentially **two separate databases**.

For example:

Order Service

| HTTP/REST

Payment Service



Payment DB

A normal local @Transactional transaction in the **Order Service** does not automatically cover the Payment Service's database.

Why

Suppose:

Order Service



Create Order 



Payment Service



Process Payment 

The Order Service's local transaction cannot simply say:

ROLLBACK Payment DB

because the Payment DB belongs to another service.

So you should not assume:

@Transactional

```
public void createOrder() {  
    orderRepository.save(order);  
    paymentService.processPayment();  
}
```

means that the Order DB and Payment DB are automatically one transaction.

They are **not normally one local database transaction**.

Simple Comparison

Scenario	Transaction
Order Service → Order DB	Local transaction
Payment Service → Payment DB	Local transaction
Order Service → Payment Service	Service-to-service communication
Order DB + Payment DB	Not normally one local transaction

In a monolithic application, you might have:

Application



One Database

A single transaction can potentially cover multiple tables.

But in microservices:

Order Service → Order DB

Payment Service → Payment DB

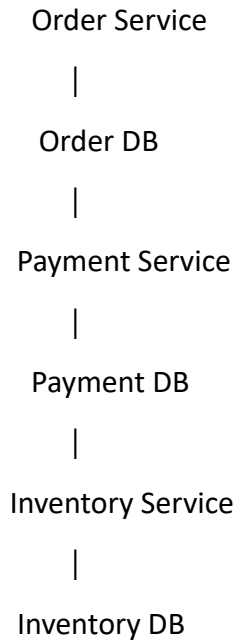
Inventory Service → Inventory DB

Each service generally **owns its own database**.

Therefore, you need distributed transaction patterns or, commonly, **event-driven/Saga-style coordination** for workflows that span services.

Example: Order + Payment + Inventory

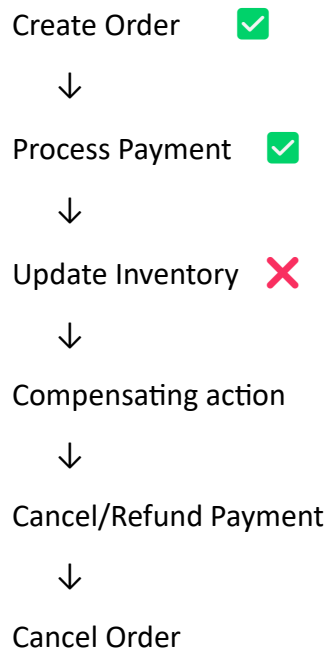
A complete order might look like:



You cannot treat all three databases as one ordinary local transaction.

Instead, the system needs to handle failures between services.

For example:



This is the kind of problem that **distributed transaction patterns such as Saga** are designed to address.

Easy Way to Remember

Local transaction

One Service



One Database



@Transactional



COMMIT / ROLLBACK

Distributed workflow

Service A



Database A



Service B



Database B

Here, a normal local @Transactional does **not** automatically provide one atomic transaction across both databases.

Step 7 — Study Distributed Transactions

A normal @Transactional transaction works well inside one service/database, but it does **not automatically make all these databases one transaction**.

1. Distributed Transaction Problem

A **distributed transaction** occurs when one business operation involves multiple services or databases.

Example: Placing an order

Order Service



Create Order 



Payment Service



Make Payment 



Inventory Service



Reduce Stock 

Now we have a problem:

- Order was created.
- Payment was completed.
- Inventory update failed.

We need a way to bring the system to a correct state.

This is the **distributed transaction problem**.

2. Two-Phase Commit — 2PC

Two-Phase Commit (2PC) is a distributed transaction protocol designed to coordinate multiple participants. It has two phases.

Phase 1 — Prepare

A coordinator asks all participating services:

Coordinator

└─→ Order DB "Can you commit?"

└─→ Payment DB "Can you commit?"

└─→ Inventory DB "Can you commit?"

Each participant responds:

Order DB → YES

Payment DB → YES

Inventory DB → YES

If everyone says **YES**, the coordinator moves to Phase 2.

Phase 2 — Commit

Coordinator

└─→ Order DB COMMIT

└─→ Payment DB COMMIT

└─→ Inventory DB COMMIT

All participants commit.

What if one says NO

For example:

Order DB → YES

Payment DB → YES

Inventory DB → NO

The coordinator can tell the participants to abort/rollback:

Order DB → ROLLBACK

Payment DB → ROLLBACK

Inventory DB → ROLLBACK

Simple idea

2PC tries to make multiple participants commit or abort together.

It provides strong coordination, but it can introduce **latency, locking, and availability/coordination complexity**, so many microservice architectures prefer asynchronous patterns instead.

3. Eventual Consistency

Eventual consistency means the different services may not show exactly the same state immediately, but they become consistent after the required events are processed.

Example:

Order Service



Order Created



Event: OrderCreated



Payment Service



Payment Processed



Inventory Service



Inventory Updated

For a short period:

Order DB → Order = CREATED

Payment DB → Payment = PENDING

Inventory DB → Old stock

Later:

Order DB → Order = CONFIRMED

Payment DB → Payment = SUCCESS

Inventory DB → Stock updated

So the system becomes consistent **over time**.

4. Compensation

In distributed systems, sometimes we cannot simply perform a database rollback across all services.

Instead, we perform a **compensating action**.

Example

Suppose:

Create Order 

Payment 

Inventory Update 

We cannot necessarily roll back the Payment DB using the Order Service's local transaction.

Instead, we can perform a compensating action:

Inventory Update → FAILED



Refund Payment



Cancel Order

So:

Original action:

Payment = SUCCESS

Compensating action:

Payment = REFUNDED

The compensation does not literally erase history; it performs a new business operation that corrects the earlier result.

5. Saga Pattern

The **Saga Pattern** is commonly used to manage long-running business workflows across microservices.

A Saga divides one large business transaction into a series of **local transactions**.

Example:

Order Service



Create Order



Payment Service



Process Payment



Inventory Service



Reserve Inventory

Each service commits its own local transaction.

If something fails, a **compensating transaction** is executed for previously completed steps.

Example

Create Order 



Process Payment 



Reserve Inventory 



Cancel/compensate Payment



Cancel Order

Two Ways to Implement Saga

1. Choreography

Services communicate through events.

Order Service

| OrderCreated

Payment Service

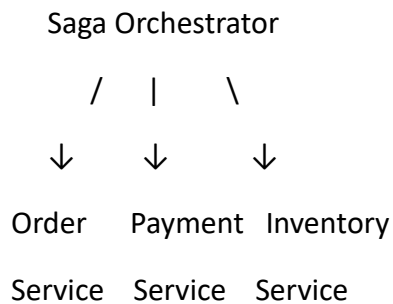
| PaymentCompleted

Inventory Service

There is no single central controller.

2. Orchestration

A central **Saga Orchestrator** controls the workflow.



The orchestrator tells each service what to do and handles failures/compensation.

Step 8 — Compare the Approaches

Local Transaction vs Distributed Transaction vs Saga

Feature	Local Transaction	Distributed Transaction / 2PC	Saga
Scope	Usually one service/database	Multiple participants	Multiple services
Main idea	One atomic DB transaction	Coordinate participants	Sequence of local transactions
Rollback	Database rollback	Coordinated abort/rollback	Compensation
Consistency	Strong within transaction	Strong coordination	Usually eventual consistency

Feature	Local Transaction	Distributed Transaction / 2PC	Saga
Complexity	Low	High	Medium
Performance	Usually efficient	Can be slower	Usually suitable for asynchronous workflows
Microservices use	Very common	Specialized cases	Common approach
Example	Order + Order DB	Order DB + Payment DB coordinated	Order → Payment → Inventory

Easy Real-World Example :Imagine ordering a mobile phone.

Local Transaction

Only Order Service:

Create Order

+

Create Order Items

↓

One Order DB transaction

↓

COMMIT / ROLLBACK

Distributed Transaction

Several databases participate:

Order DB

+

Payment DB

+

Inventory DB

↓

2PC coordination

Saga

Each service performs its own transaction:

Create Order



Process Payment



Reserve Inventory

If inventory fails:

Reserve Inventory ❌



Refund Payment



Cancel Order

Step 9 — Design Failure Scenarios

1. What happened :Imagine a customer places an order.

Step 1 — Order is created

Order Service



Create Order



Order = CREATED

The order is successfully stored.

Step 2 — Payment is attempted

Order Created



Payment Service



Payment Failed ❌

Now we have an incomplete business operation.

The customer has an order, but payment was unsuccessful.

Question 1 — What should happen to the order?

The system should normally **not leave the order as successfully confirmed**.

One possible state flow is:

CREATED



PAYMENT_PENDING



PAYMENT_FAILED



CANCELLED

For example:

Order ID: 101

Status: CANCELLED

Reason: PAYMENT_FAILED

The exact business rule depends on the application.

Another design could allow the customer to retry payment:

CREATED



PAYMENT_FAILED



PAYMENT_RETRY



Payment Successful



CONFIRMED

So the important point is to **define the order states clearly**.

Question 2 — How should the system recover?

Recovery depends on what has already happened.

Scenario A — Payment fails immediately

Create Order 

Payment 

The system can update:

Order Status = PAYMENT_FAILED

or:

Order Status = CANCELLED

No payment compensation is required because payment never succeeded.

Scenario B — Payment succeeds but inventory fails

This is more complicated:

Create Order 

Payment 

Inventory 

Now the system needs compensation.

For example:

Inventory Failed

↓

Refund Payment

↓

Cancel Order

So:

Order → CANCELLED

Payment → REFUNDED

Inventory → Not reserved

Question 3 — What data needs to be compensated?

Compensation means performing another business operation to correct a previous successful operation.

Example:

Original operation:

Payment = SUCCESS

If a later operation fails:

Inventory = FAILED

The system may perform:

Compensating operation:

Payment = REFUNDED

Similarly:

Order = CREATED

could be compensated by:

Order = CANCELLED

Example

Original actions

Create Order 

Make Payment 

Reserve Inventory 

Compensation

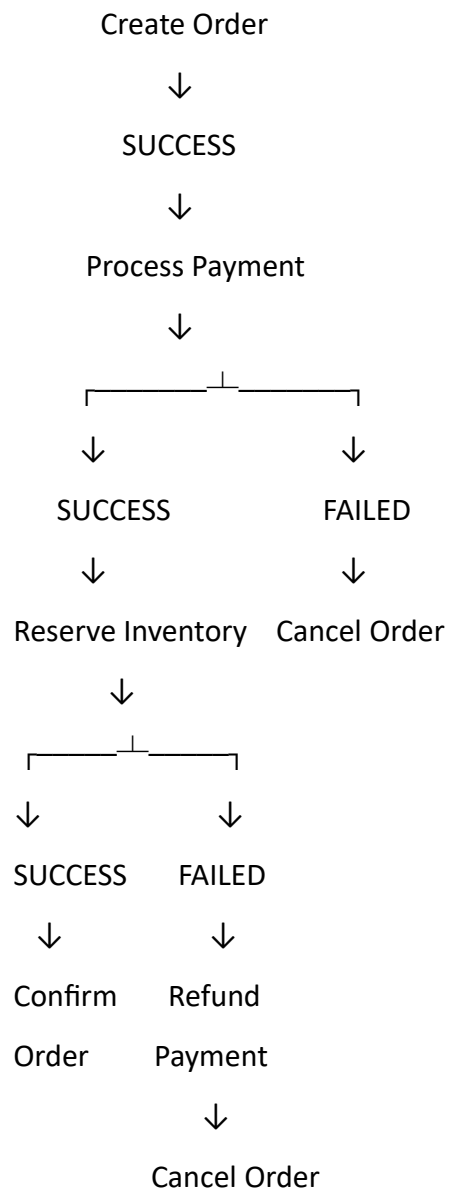
Refund Payment 

Cancel Order 

Release Inventory if applicable

Complete Failure Flow

A typical Saga-style workflow could look like this:



Step 10 — Document Findings

After designing and testing the transactions, you need to **document what you learned and implemented**.

This document should explain:

1. Transaction design

Explain which operations belong to the same transaction.

Example:

Order Service



@Transactional



Create Order

Create Order Items

Update Inventory

2. ACID properties

Document:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

Explain how your application maintains these properties.

3. Rollback behavior

Document what happens when an operation fails.

Example:

Create Order 

Create Order Item 



ROLLBACK



Order not persisted

4. Distributed transaction design

Document how services communicate.

For example:

Order Service



Payment Service



Inventory Service

Explain whether the architecture uses:

- Local transactions
- 2PC/distributed transaction coordination
- Saga
- Events
- Compensation

5. Failure scenarios

Create different failure cases.

Failure	Expected Recovery
Order creation fails	No order created
Payment fails	Mark payment failed / cancel or allow retry
Inventory fails	Refund payment and cancel order if payment succeeded
Payment service unavailable	Retry or mark payment pending
Inventory service unavailable	Keep order pending or trigger compensation
Network timeout	Retry safely / check final status